Ders 6 Çalışma Özeti

Ders 6 Çalışma Özeti

Genel Konular

- Command kalıbı (ekrandan, 00:17:10)
  - Amaç: nesnelere giden istekleri (metot çağrılarını, mesajları) nesne haline getirmek. Böylece mesajlar kuyruğa alınabilir, güncesi (logu) tutulabilir.
  - GUI programlamadan örnek: menü seçeneğine tıklanınca "şu nesnenin şu metodu çalışsın" diye kodlanır. Kalıbın önerisi, herhangi bir nesnenin herhangi bir metodu yerine belirli bir arayüzün belirli bir metodunu (`Execute()`) çalıştırmak; işin nasıl yürüyeceği bu arayüzün gerçeklemelerinde ayarlanır.
  - Dörtlü Çete'nin menü örneğinde soyut `Command` sınıfının somut alt sınıfları `OpenCommand` (dosya açar, uygulamaya `Add` ile belge ekler; bu yüzden uygulamaya geri yönlü bir bağımlılığı vardır) ve `PasteCommand` (o anda çalışılan belgeye `Paste` der) olarak anlatıldı.
  - Kalıp yapısı: `Client`, `Invoker` (bir `Command` tutar), soyut `Command` (`Execute()`), `ConcreteCommand` (`Execute()`, `state`; `Execute()` içinde `receiver->Action()`), `Receiver` (`Action()`). Client hem receiver'ı hem somut komutu tanır.
  - Invoker ile receiver aynı olmak zorunda değildir; receiver isteğe bağlıdır, somut komut işi kendi başına da yapabilir.
- Command örneği: akıllı ev kumandası (ekrandan, 00:20:00; esinlenme Head First DP)
  - `<<interface>> AcKapa` (`ac(): boolean`, `kapa(): boolean`); bunu `AydinlatmaDugmesi`, `ElektronikAlet`, `Iklimlendirme` gerçekler (her birinde `isim: String`, kurucu, `toString()`).
  - `Kumanda`: `acKapaDugmeleri: ArrayList<AcKapa>`, `dugmeEkle(AcKapa)`, `ac(dugmeNo: int): boolean`, `kapa(dugmeNo: int): boolean`, `hepsiniKapa()`. Düğme bulunamazsa `false`, bulunursa düğmenin `ac()` sonucu döner.
  - `AnaProgram`: `evBoskenSokakKapisiAcildi()` antre ışıklarını ve kombiyi açar; `evdeKimseKalmadi()` bütün düğmeleri kapatır.
- Iterator kalıbı (ekrandan, 00:30:00)
  - Amaç: parçalardan oluşan bir bileşenin parçalarını, bileşenin iç yapısını açığa çıkarmadan, istenilen bir biçimde dolaşmak. Sorun: veri yapısının arayüzünü birçok farklı dolaşım türüyle şişirmek istemeyiz.
  - Farklı dolaşım biçimleri: ikili ağaçta LDR, DLR, LRD gibi sıralar; tek yönlü, çift yönlü ve dairesel bağlı liste.
  - Her veri yapısı için dolaşma kodunu ayrı yazmak bakım kâbusudur: bir hatayı bütün kopyalarda düzeltmek gerekir.
  - Java'da her `Collection`'dan bir `Iterator` istenebilir; bu, kalıba tam uymaz, daha basittir, ama çoğu durumda yeter (KISS).
- Iterator çözümleri ve kalıp yapısı (ekrandan, 00:32:15, 00:40:00, 00:36:50)
  - 1. çözüm: `List` (`Count()`, `Append(Element)`, `Remove(Element)`) ve `ListIterator` (`First()`, `Next()`, `IsDone()`, `CurrentItem()`, `index`). Güçlü yönü: veri yapısı ile dolaşım biçimi ayrıldı. Zayıf yönü: belli bir veri yapısı ile belli bir dolaşım biçimi bağlı.
  - 2. çözüm: soyut `AbstractList` (`CreateIterator()`, `Count()`, `Append(Item)`, `Remove(Item)`) ve soyut `Iterator`; `List`–`ListIterator`, `SkipList`–`SkipListIterator` çiftleri. Somut liste ile kendi iterator'ı arasındaki bağ yine kalır; daha esnek işler için ileride Visitor kalıbı görülecek.
  - Kalıp yapısı: `Aggregate` (`CreateIterator()`), `ConcreteAggregate` (`return new Concre-`

teIterator(this)), Iterator (First, Next, IsDone, CurrentItem), ConcreteIterator. java.util paketindeki veri yapıları bu kalıbı kullanır.
  - Hoca iterator için Türkçe "gezgin" sözcüğünü kullandı.
- Iterator gerçekleme ayrıntıları (ekrandan, 00:38:15)
  - Dolaşırken değiştirmeye izin verilecek mi? Iterator nesnelerin kendisini gösterirse değişiklik mümkündür; klonları verilirse değişikliğe izin verilmemiş olur.
  - Dolaşım algoritmasını kim gerçekleyecek, parçalar birer birer mi hepsi birden mi verilecek? Ya istekçi gezgine komut verme sırasını belirler ve parçaları tek tek okur, ya da istekçi bir dolaşım istemi verir ve gezgin parçaları bir dizide döndürür.
  - Dolaşım sırası (örneğin LDR) iterator istenirken bir enum ile verilebilir; bu seçim veri yapısında ya da iterator'da kodlanabilir.
- Iterator örneği: yemek siparişi menüleri (ekrandan, 00:44:40, 00:56:15; kaynak Head First DP)
  - Biri kahvaltılıkta (PancakeHouseMenu, ArrayList kullanır), biri yemekte (DinerMenu, en fazla 6 elemanlı dizi kullanır) uzman iki firma alındı. MenuItem (ad, açıklama, vejetaryen mi, fiyat) ortak.
  - Acemi çözüm (WaitressQuickAndDirty): iki menüyü üye alan olarak tutar, dizide ve listede ayrı ayrı dolaşır; vejetaryen menü için yine ayrı döngüler gerekir. Her yeni firma yeni kod bloğu demektir.
  - Iterator çözümü: <<interface>> Menu (createIterator(): Iterator<MenuItem>), Iterator<MenuItem> (hasNext(): boolean, next(): MenuItem), DinerMenuIterator, PancakeHouseMenuIterator. Waitress iki menüyü kurucuda alır; dolaşma tek bir printMenu(Iterator) metodunda toplanır.
- Memento kalıbı (ekrandan, 01:30:00)
  - Amaç: bir nesnenin durum bilgisini (ya da bir alt kümesini) sarmalama ilkesini bozmadan elde etmek.
  - Kullanım: ikincil saklama ortamına yazmak, ağ üzerinden aktarmak, geri alma (undo/redo), hareket (transaction) mantığı (adımlardan biri başarısızsa nesne başlangıçtaki güvenli haline döner). Yalnız ilgilenilen alt küme saklanırsa bellek tasarrufu sağlanır.
  - Yapı: Originator (SetMemento(Memento m), CreateMemento(), state; return new Memento(state) ve state = m->GetState()), Memento (GetState(), SetState(), state), Caretaker (mementoları tutar; liste, map, ağaç ya da yığın olabilir).
  - Kalıp bileşenleri: Originator durum bilgisi saklanacak nesne; Memento durum bilgisinin gerekli kısımlarını saklar; Caretaker gerekli anlarda kopya alır ve kopyalardan birini geçerli duruma getirir.
- Memento gerçekleme seçenekleri (ekrandan, 01:50:00, 02:00:00)
  - C++: Memento sınıfında friend Originator bildirimi.
  - Java, paket erişimi: Memento metotlarına erişim belirleyici yazılmaz, Originator ile Memento tek bir pakette tutulur. Jar dosyası paketler mühürlenerek (sealed) oluşturulmazsa başkası pakete sınıf ekleyip bu metotlara erişebilir.
  - Java, iç sınıf: Memento, Originator'ın public iç sınıfı yapılır, getState/setState private olur. createMemento içinde durum klonlanmalıdır (State implements Cloneable).
- Memento örneği: belge işleme programında undo/redo (ekrandan, 02:02:20)
  - Document (currentState: DocumentMemento, states: List<DocumentMemento>, position: int; undo(), redo(), anyOperation()), DocumentMemento (documentState), Controller.
- Observer kalıbı (ekrandan, 02:07:50, 02:10:00, 02:20:00)
  - Amaç: bir nesnenin durumu değişince onunla ilişkili bütün nesnelerin (ya da bir alt kümesinin) haberdar edilmesi ve güncellenmesi. İlişkiler çalışma anında ekleme ve çıkarmayla kurulur.
  - Örnek: hesap tablosundaki veri değişince o veriden çizilen grafiklerin kendiliğinden güncellenmesi.
  - Yapı: soyut Subject (Attach(Observer), Detach(Observer), Notify(); for all o in observers { o->Update() }), ConcreteSubject (GetState(), SetState(), subjectState), soyut Observer (Update()), ConcreteObserver (Update(), observerState = subject->GetState()). ConcreteObserver'dan ConcreteSubject'e giden subject başvurusu isteğe bağlıdır.
  - Pull ve push modeli, statik gözleyici listesi, birden fazla nesneyi gözleme, haber

verme mesajının tetiklenmesi.
- Observer örneği: müşteri ilişkileri programı (ekrandan, 02:24:20)
  - Yeni müşteri eklenince hoş geldiniz mektubu gönderilmeli ve adres doğrulanmalı; sonra pazarlamacılar en yakın fiziksel şubeye 10 km yakın müşteriler için mektuba indirim kuponu eklenmesini istiyor.
  - `Customer (myObs: Vector<ICObserver> {static}, attach/detach {static}, getState(), notifyObs()), <<interface>> ICObserver (update(myCust: Customer))`, `AddrVerification`, `BrickAndMortar`, `WelcomeLetter`.

Hocanın Özellikle Vurguladığı Kısımlar

- Aynı sınıf yapısına sahip birçok kalıp vardır
  - Bir arayüz, onu gerçekleyen sınıflar ve arayüzü kullanan bir sınıftan oluşan yapı birçok kalıpta görülür. Bu kalıpları ayıran şey işlerin yürütülme ayrıntıları ve kalıpların amaçlarıdır.
  - Hoca bu yapıdaki kalıpların adlarının akılda tutulmasını, bir problemde bunlardan birinin en uygun, diğerlerinin "yarı uygun" diye savunulabileceğini "önemli bir ipucu" olarak söyledi.
- Command'ın özü
  - Önemli olan komutları nesnelere çevirebilme fikridir; kodun geri kalanı basittir.
- KISS ilkesi
  - "Keep it simple, stupid": Java'nın basit `Iterator`'ı işinizi görüyorsa kalıbı baştan kurmayın. Basit demek kötü demek değildir.
- Iterator çözümünde geriye uyumluluk
  - `Waitress`'ın kurucusuna menüler yerine doğrudan iterator'lar verilebilirdi, ama dışarıya verilen arayüzü değiştirmemek, geriye uyumluluğu korumak için menüler verildi.
  - Dolaşma döngüsünün `printMenu` ve `printVegetarianMenu` içinde benzer olması kopyala-yapıştır sayılmaz; değişmeyecek basit bir dolaşma kodudur.
- Memento'da güvenlik
  - Memento'nun `GetState`/`SetState` metotlarına Caretaker ya da başka bir sınıf erişememeli, yalnız Originator erişebilmeli; aksi halde dışarıdan sahte bir durum verilebilir.
  - Paket erişimi yeterince güvenli değildir. İç sınıf çözümünde durum klonlanmazsa test hatalı sonuç verir.
  - Aynı sınıftan başka bir Originator'dan alınmış ("sahte") bir mementonun verilmesi iç sınıf çözümüyle de engellenemedi; hoca bunun tam bir çözümünü bulamadığını açıkça söyledi.
- Observer'da Subject soyut sınıf olmalı
  - Gözleyici ekleme, silme ve haber verme işlevleri bir kez kodlansın diye `Subject` arayüz değil soyut sınıftır.
  - Bağlaşım ya bir yönde ya öbür yönde mutlaka olacaktır; iki yönde birden olmaz. Push ve pull modeli arasında kâr-zarar analizi yapılır.

Kısa Tekrar Notları

- Command: istekleri nesneye çevirir; `Invoker` → `Command.Execute()` → `ConcreteCommand` → (isteğe bağlı) `Receiver.Action()`.
- Akıllı ev: `AcKapa` arayüzü ortak `ac()`/`kapa()`; `Kumanda` düğmeleri `ArrayList<AcKapa>` içinde numarayla tutar.
- Iterator: iç yapıyı açmadan dolaşma; `Aggregate.CreateIterator()` ve `Iterator` (`First`, `Next`, `IsDone`, `CurrentItem`).
- Iterator gerçeklemesinde kararlar: dolaşırken değişiklik, dolaşımı kimin yöneteceği, birer birer mi topluca mı.
- Menü örneği: dizi ve `ArrayList` farkı iterator arkasında gizlendi; `Waitress` iki menüyü aynı biçimde dolaşır.
- Memento: Originator (`CreateMemento`, `SetMemento`), Memento (`state`), Caretaker (mementoları saklar).
- Java'da Memento: paket erişimi (mühürlü jar gerekir) ya da iç sınıf (durum klonlanmalı).

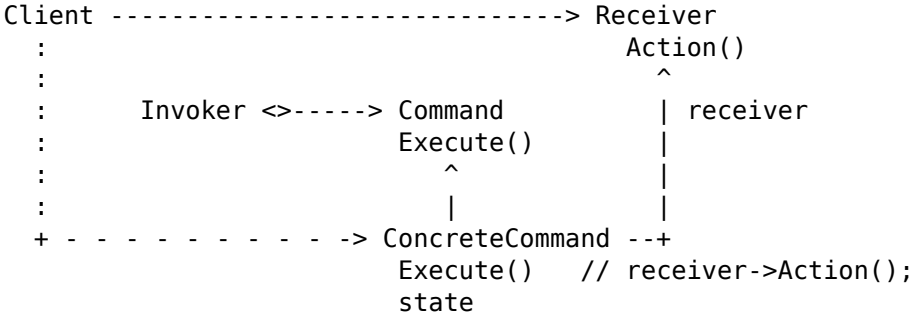
- Undo/redo: `position` azaltılmadan önce sıfırın altına düşmemesi denetlenmeli.
- Observer: `Attach`, `Detach`, `Notify` → her gözleyicinin `Update()`'i.
- Pull: gözlenen yalnız haber verir, gözleyici durumu sorar. Push: gözlenen ayrıntılı bilgiyi gönderir.
- Müşteri örneğinde `update(Customer)` müşterinin kendisini gönderir; hoca bunu push modeli olarak niteledi. Yalnız müşteriler izlenecekse soyut `Subject` gerekmez.

Detaylı Açıklamalar

Command

Hoca kalıbı GUI programlamadaki menülerle açıkladı. Bir uygulamanın menüsüne öğeler eklenir ve her öğenin bir tıklanma olayı vardır. Yalnız tıklanınca çalışacak kodu değiştirmek isteniyorsa çerçevenin (framework) `click` metodunun içini değiştirmeye izin vermesi yeter. Ama birden fazla komut söz konusuysa ve komutları kuyruğa almak, güncesini tutmak gibi ek işler isteniyorsa `Execute` metodu olan bir `Command` arayüzü kurulur. Menü öğesine tıklanınca çalışabilecek çok farklı komutlar yalnız bu ortak arayüzü gerçekledikleri için yürütülebilir. Böylece metot bir nesne haline gelir: "komut nesne oldu."

Kalıp yapısı (ekrandan, 00:17:10):

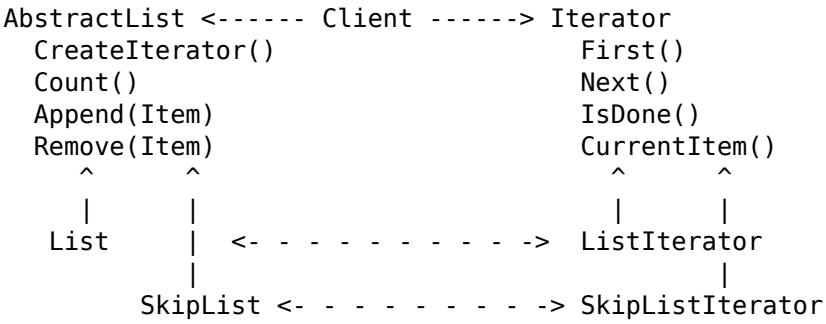


Akıllı ev örneğinde (ekrandan, 00:20:00) hoca kumanda panelini sıfırdan başlayan düğme numaralarıyla kullanmak için `ArrayList` seçtiğini söyledi. `ac(dugmeNo)` düğmeyi bulamazsa `false` döndürür, bulursa düğmenin `ac()` metodunun döndürdüğünü iletir. Açık bir düğmeyi yeniden açmaya çalışınca ne olacağı gibi ayrıntılar iskelet kodda ele alınmadı.

> **Ek bilgi (derste anlatılmadı):** Örnekte `AcKapa` arayüzü `Command`, `ac()`/`kapa()` metotları `Execute()`, `Kumanda` ise `Invoker` rolündedir. Somut sınıflar işi ayrı bir receiver'a vermeden kendileri yaptığı için örnekte receiver yoktur; bu, hocanın "receiver isteğe bağlıdır" sözüyle uyumludur.

Iterator

Iterator'ın 2. çözümünün sınıf şeması (ekrandan, 00:40:00):



Menü örneğinde (ekrandan, 01:00:00) `Waitress` içindeki yardımcı metot her öğenin adını, fiyatını ve açıklamasını yazar:

```
private void printMenuItem( MenuItem menuItem ) {
    System.out.print(menuItem.getName() + ", ");
    System.out.print(menuItem.getPrice() + " -- ");
```

```
    System.out.println(menuItem.getDescription());
}
```

Ana program (ekrandan, 01:20:00):

```
public static void main(String[] args) {
    Waitress w = new Waitress(new PancakeHouseMenu(), new DinerMenu() );
    //WaitressQuickAndDirty w = new WaitressQuickAndDirty( );
    w.printMenu();
}
```

Hoca firma sayısı arttığında `Waitress` kurucusunun imzasının her seferinde değişmemesi için kurucunun değişken sayıda `Menu` parametresi (Java'da `...` sözdizimi, C'deki `printf` gibi) alabileceğini söyledi.

> **Ders sonrası not:** Hoca değişken sayılı parametrelerin Java 1.6 ile geldiğini tahmin etti. Bu özellik büyük olasılıkla Java 5 (1.5) ile gelmiştir.

Memento

İç sınıf çözümündeki test (ekrandan, 02:00:00):

```
package dp.mementoV2Outer;
import dp.mementoV2.innerClass.*;
import dp.mementoV2.innerClass.Originator.Memento;

public class Caretaker {
    public void test( ) {
        Originator subject = new Originator( );
        subject.modify("ilk durum");
        Memento mem = subject.createMemento( );
        subject.modify("yeni durum");
        //bir süre sonra...
        subject.setMemento( mem );
        System.out.println(subject);
    }
    public static void main( String[] args ) {
        Caretaker ct = new Caretaker();
        ct.test();
    }
}
```

Çıktı `ilk durum` olur (ekrandan, 01:50:00). Hoca durum klonlanmadığında çıktının hatalı olarak `yeni durum` olduğunu söyledi (01:54:56).

> **Ek bilgi (derste anlatılmadı):** Klonlama yapılmazsa memento, Originator'ın kullandığı `State` nesnesinin kendisine bir başvuru tutar. `modify` bu ortak nesneyi değiştirdiği için mementodaki durum da değişir ve geri dönüş işe yaramaz. Klon, o anki durumun ayrı bir kopyasını saklar.

Hoca sahte memento sorununa iki fikir söyledi: mementoların saklanmasını Caretaker yerine Originator'a vermek ve Caretaker'a `activateMemento(int siraNo)` gibi bir metot sunmak (ama bu, işi Caretaker'a vermek isteyen kalıbın özüne ters düşer), ya da mementonun içine onu üreten nesnenin kimliğini (`hashCode`) gömüp yalnız kendi ürettiği mementoyu kabul etmek. İkinci çözüm yalnız program çalıştığı sürece geçerlidir; diske saklanan mementolar program yeniden açıldığında yeni nesnelerin `hashCode` değerleri farklı olacağı için geçersiz kalır.

Undo/redo örneğindeki slayt kodu (ekrandan, 02:02:20):

```
public void undo( ) {
    position -= 1;
    currentState = states.get(position);
}
public void redo( ) {
    position += 1;
```

```
    currentState = states.get(position);
}
public void anyOperation( ) {
    states.add(currentState);
}
```

Hoca `position`'ın sıfırın altına düşmemesi için `undo` içinde `if (position > 0) position--;` biçiminde bir denetim gerektiğini söyledi. Slayttaki yoruma göre bu örnek "biraz yarım" bir Memento uygulamasıdır, çünkü Caretaker rolünü `Document` üstlenmiştir.

Observer

Durum bilgisinin gözleyicilere gönderilmesi (ekrandan, 02:10:00):

- Pull modeli: gözlenen yalnız haber verir, gözleyiciler durum bilgisini gözlenenden alır. Gözlenenin gözleyicilerle bağlantısını azaltır, gözleyici durumun yalnız istediği kısmını alabilir; ama gözleyici gözlenende nelerin değiştiğini kendisi çıkarmak zorunda kalır ve gözleyiciden gözlenene bir başvuru gerekir.
- Push modeli: gözlenen, durum değişikliği hakkında ayrıntılı bilgi gönderir. Mesaj boyutu artar.

Hoca bunu IoT ile örnekledi: GSM üzerinden bağlı hafif bir sensörde gönderilen ve alınan her bayt masraftır ve basit bir sensöre ayrıntılı bilgi gönderecek kod yazılamayabilir. Bu durumda işi gözleyiciye bırakan model seçilir.

> **Ders sonrası not:** Hoca 02:13:47 civarında hafif gözlenenler için seçilecek modeli bir an "push model" diye adlandırdı. Anlatılan durum (gözlenen yalnız haber verir, işi gözleyici yapar) slayttaki tanıma göre pull modelidir.

Haber verme mesajının tetiklenmesi (ekrandan, 02:20:00): gözleyiciler tetiklerse gözlenenin istekçilerini kodlayan kişi tetikleme komutunu vermeyi unutabilir; gözlenen tetiklerse peş peşe durum değişikliklerinde tek mesaj yerine birçok gözleyiciye bir sürü mesaj gider. Hoca çok hassas ayarlanmış bir sıcaklık sensörünün küçük değişikliklerde sürekli güncelleme göndermesini örnek verdi. Zaman aşımı (timeout) kullanmak, aralık doğru seçilmezse ya gereksiz mesaj üretir ya da önemli bir değişikliği kaçırır. Gözleyicinin ara sıra gözleneni sorgulaması ise gözleyicide gözlenene başvuru tutmayı, yani ek bağlaşımı gerektirir.

Bir gözleyici birden fazla nesneyi gözleyebilir ve gözlenenin farklı türden değişikliklerini farklı gözleyiciler izleyebilir. Hoca bir pencereyi örnek verdi: bir gözleyici yalnız pencerenin odak kazanıp kaybetmesiyle, bir başkası küçültülüp büyütülmesiyle, bir başkası diğer olaylarla ilgilenir.

Kalıbın kullanılabileceği anlar (ekrandan, 02:20:00): bir soyutlamanın birbirine bağımlı birden fazla parçası olduğunda; bir nesnedeki değişikliklerin derleme anında bilinmeyen sayıda nesneyi etkilemesi gerektiğinde; bağlaşımı arttırmadan bir nesnenin diğerlerini bir durumdan haberdar etmesi istendiğinde.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.